



Courses >

EAG
V3 >

Session 7 - Memory and Retrieval: Embeddings, FAISS, and Indexed Knowledge



Table of Contents



Session 7 - Memory and Retrieval: Embeddings, FAISS, and Indexed Knowledge

Session 6 left us with a four-role agent that read its memory by keyword overlap/search. **Session 7** adds vector retrieval underneath that interface. The architecture from Session 6 is preserved without revision. `MemoryItem` gains one optional field. The Memory service learns one new code path. Two new MCP tools added for **document ingestion** (New in V3) and **vector search** to the model. The gateway gains one endpoint. Those are the only changes!

The changes is modest. Session 6 was already a heavy lift. Session 7 is a one-step extension that turns the same agent into a retrieval-aware one. *Hybrid retrieval with reciprocal rank fusion, cross-encoder reranking, semantic chunking, and graph-aware retrieval all wait for later sessions where they can each receive proper treatment* What is added here is the smallest amount of

[Mark Complete](#)

vector science that justifies a real RAG demonstration and exposes us to FAISS as the canonical vector index.

The session also documents a small but consequential moment of architectural drift and the correction that followed (intentionally to replicate how things would surface if you were in production, this is taken from V1/V2/V3 actual code development experience). The first implementation of the new tools (would) accidentally moved tool-selection responsibility from Decision into Perception. The eight (four new added for S7) queries passed; but the role boundary was breached. The discovery (as you'd later in your journey), the diagnostic discipline that surfaced it (how to find it), and the diff that restored the boundary are part of the session's substance (sticking to the principle of *separation of concerns*). They illustrate a class of bug that this course (and your production cycles) will see again, and a habit of inquiry that prevents it.

Where the course stands

By the end of Session 6 the agent had **four named roles** communicating through Pydantic contracts. Memory was a typed service with keyword search. Perception ran every iteration and maintained the goal list. Decision chose between answering and dispatching one MCP tool call. Action ran the dispatch.

term, and its actually not. In production you'll delete is sooner). The whole system was wired to the LLM gateway introduced in Session 5.

The four target queries in that session allowed for artifact attachment, multi-goal decomposition with memory carryover, durable preferences across runs, and multi-source synthesis. The queries worked because the agent could see, plan, dispatch, and remember. They did not require any deeper retrieval than keyword search/overlap.

Session 7's queries require more. Indexing a corpus of research papers and answering questions whose surface tokens do not appear in any single chunk is the workload that justifies an embedding model and a vector index. The agent that ran the Session 6 queries cannot answer "How do these papers handle the credit assignment problem?" against a corpus that talks about backpropagation through reasoning steps and reward shaping. The phrase "credit assignment" appears in none of the chunks. Keyword search returns nothing. Vector retrieval, given an embedding of the query and embeddings of the chunks, surfaces relevant material from four out of five documents in the indexed corpus. That demonstration is the architectural upgrade for Session 7.

What this session adds

Four changes. Each is local. None requires modifying the agent loop or the Pydantic contracts developed in Session 6.

The first change is to the gateway. A new endpoint, `POST /v1/embed`, accepts a piece of text and returns a 768-dimensional vector. The default provider is Ollama with the `nomic-embed-text` model. A fallback provider is configured via environment variables; the current deployment uses Gemini's `gemini-embedding-001` with `outputDimensionality=768` so the fallback vector lives in the same space as the default. The endpoint exposes nothing about the agent's prompts to the embedding model; it is a deterministic function from text to vector.

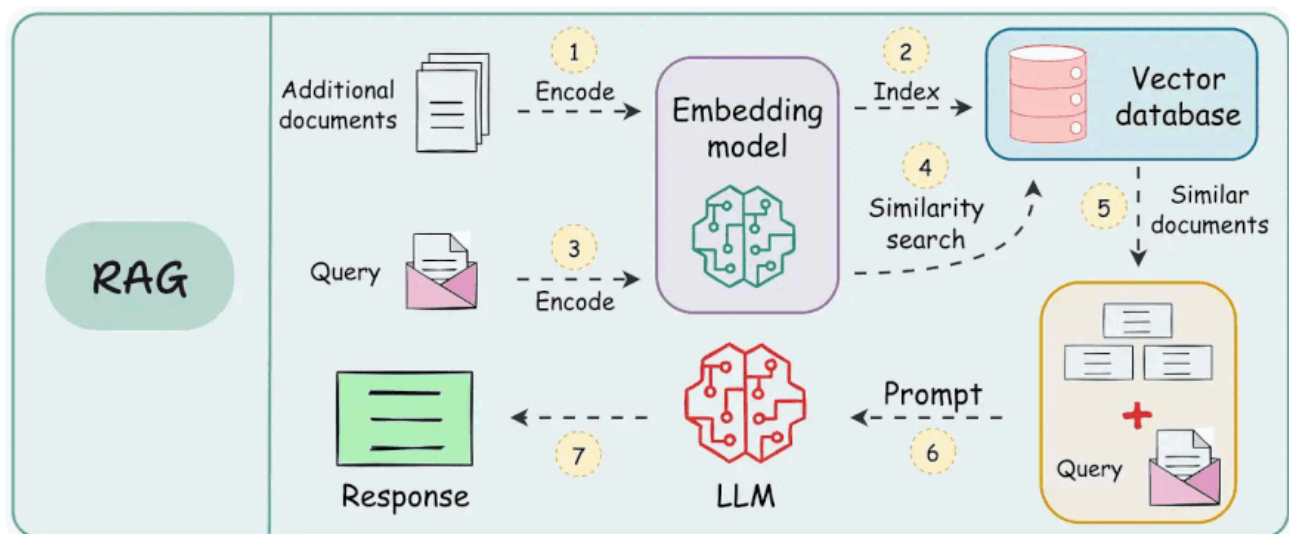
The second change is to the `MemoryItem` schema. One optional field, `embedding: list[float] | None`, is added. Memory writes for items of kind `fact`, `preference`, and `tool_outcome` compute an embedding at insert time and store it on the item. Scratchpad items skip embedding.

The third change is to the Memory service. Reads first run a vector search through FAISS over the items that carry embeddings. When the vector path returns at least one hit, the read returns those items. When the vector path returns nothing, the read falls back to the keyword overlap from Session 6. The vector path dominates in practice once any embeddings exist on disk. The keyword fallback exists for cold-start runs and for partial-availability cases where the gateway's embed endpoint is unreachable.

The fourth change is to the MCP server. Two new tools, `index_document(path)` and `search_knowledge(query, k)`, expose the same machinery to the model. The first tool reads a sandbox file or an artifact, chunks it with a sliding window, embeds each chunk, and writes the chunks into Memory as `fact` items. The second tool runs the Memory read path explicitly when Decision wants to query the indexed corpus without waiting for the implicit retrieval inside Perception's

The full set of changes is around two hundred lines of code across five files. Schemas, contracts, and the agent loop are unchanged. Session 6's eight failure-mode safety nets remain in place. Students who built a working Session 6 agent can land Session 7 without rewriting any role.

RAG



Embeddings as a vector

An embedding is a fixed-length numerical vector that represents the meaning of a piece of text. The vector is produced by a neural network trained to put semantically similar texts close together in vector space and dissimilar texts far apart. The geometry of the space carries the semantic structure. Two paragraphs about photosynthesis sit near each other; a paragraph about photosynthesis and a paragraph about cricket sit far apart. The distance metric most commonly used for text embeddings is cosine similarity, which measures the angle between two vectors and ignores their magnitudes.

The model used in this session is `nomic-embed-text`, run locally through Ollama. The model produces a 768-dimensional vector for each input. The dimensionality is the same for every input, regardless of text length. A one-word query and a thousand-word document both come back as 768 floats. The vector is not interpretable position by position; the 412th element has no semantic name. The space is interpretable only relationally, through distances and angles.

A small similarity matrix illustrates the geometry. Given four sentences, the model produces four vectors. The cosine similarity between two vectors is a number in the interval from negative one to positive one, where one is identical, zero is orthogonal, and negative one is opposite.

	cosine similarity
"How does AlphaFold work?"	
vs "How do proteins fold?"	0.78
vs "What is the capital of France?"	0.18
vs "Explain how neural networks learn."	0.42

vs "What is the capital of France?"0.15

vs "Explain how neural networks learn."0.39

The values above are illustrative; running the same comparison with `embeddings_compare_ollama.py` ([FILE](#)) from the Session 7 reference code on a particular Ollama install produces numbers within a few percentage points. The structure holds. Questions about protein folding and AlphaFold cluster. Questions about neural networks sit at moderate distance from both. The unrelated capital-of-France query sits far from everything.

Embeddings come in two families. The model used here is a dense embedding model: every position in the output vector carries a real number, and the vector has no zeros. Sparse embedding models produce vectors of higher dimension where most positions are zero and the non-zero positions correspond to specific tokens or learned features.

Dense Embedding vs Sparse Embedding

FEATURE	DENSE EMBEDDING	SPARSE EMBEDDING
Captures meaning	Strong	Weak
Captures exact words	Medium/Weak	Strong
Vector size	Small fixed size	Very large
Mostly zeros	No	Yes
Good for RAG	Yes	Yes, as hybrid
Handles synonyms	Good	Poor
Handles codes/IDs	Weak	Strong

BM25 or *Best Matching function number 25*

BM25 is the most familiar sparse representation; SPLADE and similar learned-sparse models are recent additions. Dense embeddings excel at semantic similarity and synonymy. Sparse embeddings excel at exact-token recall and at queries that hinge on a rare keyword. Production retrieval systems often combine the two through a fusion algorithm such as Reciprocal Rank Fusion, in which each retriever returns a ranked list and the fused score for a document is the sum across retrievers of one over k plus the rank. The constant k is small, commonly sixty. Documents that rank highly in either retriever rise in the fused list.

Dense Embedding vs BM25

FEATURE	BM25	DENSE EMBEDDING
Search type	Keyword / lexical	Semantic
Understands synonyms	Weak	Strong
Handles exact terms	Strong	Medium/weak

FEATURE	BM25	DENSE EMBEDDING
Needs training		No
		Usually yes/pretrained
Good for technical docs	Very good	Good but risky alone

In RAG, best answer: use hybrid retrieval

For serious RAG, use both:

```
Dense retrieval + Sparse retrieval + Reranker
```

Architecture:

```
User Query
  ↓
Dense Search: finds semantic matches
Sparse Search: finds keyword/exact matches
  ↓
Merge results
  ↓
Reranker
  ↓
Top chunks sent to LLM
```

Example:

```
Query:
"What does ASTM B117 say about salt spray corrosion?"

Dense search may retrieve:
"marine corrosion testing procedures"
```

Hybrid retrieval gives both.

Practical rule

Use **dense embeddings** when your query is conceptual:

"Why do solar supports corrode near the sea?"

Use **sparse embeddings** when your query has exact terms:

"ASTM B117 HDG Z275 FRP 6061-T6"

Use **hybrid** for production RAG.

Session 7 uses dense embeddings only. The vocabulary above is provided so you can read papers and production systems that combine the two paths. The implementation of hybrid retrieval, with both a dense index and a sparse index and a fusion step, waits for a later session.

FAISS

FAISS is the library this course uses for vector indexing. The name expands to Facebook AI Similarity Search. The library is a C++ implementation with a Python binding that handles dense vectors at a scale this course will not approach for several sessions. For Session 7 the relevant subset is small.

A FAISS index is an in-memory data structure that stores vectors and supports nearest-neighbor search. The index is constructed for a fixed dimension. Adding a vector of a different dimension to an existing index raises an error. The session pins the embedding model precisely so that this never happens silently.

The simplest index type is `IndexFlatL2`, which stores vectors in their natural form and computes the Euclidean distance between the query and every stored vector at search time. Search is exact but scales linearly in the size of the index. The session uses a close relative, `IndexFlatIP`, which computes the inner product rather than the Euclidean distance. When the vectors stored in the index and the query vectors are both L2-normalized, the inner product equals the cosine similarity exactly. The session normalizes every vector at insertion and at query time so that cosine similarity is what FAISS actually returns.

A canonical interaction with the index has four steps. The index is constructed with a known dimension. Vectors are added by calling `add` with a two-dimensional numpy array. A query vector is passed to `search` along with a value of `k`. The library returns the top `k` vectors ranked by similarity, along with their similarity scores. FAISS stores vectors by their order of insertion. The integer position returned by `search` is the position at which the vector was added. The application is responsible for maintaining a parallel mapping from the integer position back to whatever identifier the application uses. In the Session 7 implementation, the mapping is a `list[str]` of `MemoryItem.id` values; the parallel list is serialized to disk alongside the index file.

the pair back on startup is one call to `faiss.read_index` and one `json.loads` . The session keeps these two files under `state/index.faiss` and `state/index_ids.json` . They are read at every Memory invocation. The reload cost is modest at the scale this session reaches; rebuilding the index entirely from `memory.json` on cold start takes under one second for the five-paper corpus.

FAISS is a similarity index. There is no query language, no metadata filter at the FAISS layer, no transaction model; FAISS is responsible for finding nearest neighbours in vector space and for nothing else. Filtering by metadata happens in Python after FAISS returns its `k` candidates. For workloads larger than a single student's laptop, the session does not change. FAISS supports compressed indices, sharded indices, GPU indices, and several approximate-nearest-neighbor algorithms; none of those are required here.

FAISS SELF LEARNING NOTES

Talk to ChatGPT/Gemini/Claude etc about FAISS and learn more about the below section

FAISS stands for:

Facebook AI Similarity Search

It is a library for doing **fast vector similarity search**.

In RAG, FAISS is commonly used to store and search **dense embeddings**.

Simple meaning

```
"FRP support for solar panels"  
→ [0.12, -0.44, 0.09, ...]
```

```
"corrosion resistant mounting structure"  
→ [0.10, -0.41, 0.13, ...]
```

FAISS helps you quickly find:

"Which stored vectors are closest to this query vector?"

So when a user asks:

```
"What material should I use for solar panel supports in salty areas?"
```

You embed the query, then FAISS searches the stored document embeddings and returns the closest chunks.

Where FAISS fits in RAG

Typical RAG pipeline:

```
Documents  
↓  
Split into chunks  
↓  
Create embeddings  
↓  
Store embeddings in FAISS  
↓
```

```
Create query embedding
↓
Search FAISS
↓
Retrieve top matching chunks
↓
Send chunks to LLM
↓
Final answer
```

FAISS is the **vector search engine** part.

What FAISS stores

FAISS stores vectors like this:

```
chunk_1 → [0.21, -0.11, 0.87, ...]
chunk_2 → [0.05, 0.72, -0.31, ...]
chunk_3 → [-0.44, 0.18, 0.09, ...]
```

But usually you separately keep metadata:

```
chunk_id
document_name
page_number
original_text
source_url
```

FAISS itself mainly handles the vector search. Your application usually maps the returned vector IDs back to text chunks.

What FAISS is good at

```
10,000 chunks  
1 million chunks  
100 million chunks
```

Instead of comparing the query vector with every stored vector slowly, FAISS uses optimized indexing methods.

Similarity search

FAISS can search using metrics like:

1. L2 distance

Measures geometric distance:

```
smaller distance = more similar
```

2. Inner product

Often used for cosine-style similarity if vectors are normalized:

```
larger score = more similar
```

3. Cosine similarity

Usually done by normalizing vectors first, then using inner product.

```
faiss.normalize_L2(embeddings)  
faiss.normalize_L2(query)
```

Common FAISS index types

1. IndexFlatL2

Exact search.

Slow for huge data, but accurate.

Good for small/medium datasets.

```
index = faiss.IndexFlatL2(dimension)
```

2. IndexFlatIP

Exact inner-product search.

Often used with normalized embeddings.

```
index = faiss.IndexFlatIP(dimension)
```

3. IndexIVFFlat

Approximate search using clusters.

Faster, but needs training.

Good for larger datasets.

4. IndexHNSWFlat

Very fast search, strong recall.

5. IndexIVFPQ

Compressed approximate search.

Useful when you have massive vector collections and memory pressure.

FAISS vs BM25

FEATURE	FAISS	BM25
Retrieval type	Dense vector search	Sparse keyword search
Captures semantic meaning	Strong	Weak
Captures exact words	Medium/weak	Strong
Needs embeddings	Yes	No
Good for synonyms	Yes	No
Good for codes/IDs	Risky	Strong
Example query	"salt damage near sea"	"ASTM B117 Z275 HDG"

For production RAG, use both:

FAISS dense search + BM25 sparse search + reranker

```
import faiss
import numpy as np

# Suppose each embedding has 384 dimensions
dimension = 384

# Create sample document embeddings
embeddings = np.random.random((1000, dimension)).astype("float32")

# Create FAISS index
index = faiss.IndexFlatL2(dimension)

# Add vectors to index
index.add(embeddings)

# Create query embedding
query = np.random.random((1, dimension)).astype("float32")

# Search top 5 nearest vectors
distances, ids = index.search(query, 5)

print(ids)
print(distances)
```

Output:

```
ids          → IDs of closest chunks
distances    → similarity/distance scores
```

Then your app uses those IDs to fetch the original text chunks.

Practical mental model

Think of FAISS as:

Not a full database.

It does not automatically understand documents, pages, metadata, citations, permissions, or chunking. It just helps you do fast nearest-neighbor search over vectors.

For RAG, FAISS is usually one component inside a larger retrieval system.

Chunking for retrieval

An embedding model produces one vector per input. A book and a one-line definition both come back as 768 floats. The longer the input, the more semantic content gets averaged into the single vector, and the less precise retrieval becomes. The standard remedy is chunking. The document is split into shorter passages and each passage is embedded separately. Retrieval returns a chunk rather than a whole document.

The chunker in `index_document` uses a sliding window over words. The default chunk size is 400 words with an 80-word overlap. The overlap matters because semantic units rarely align with arbitrary word counts. A sentence that explains the key idea of a section may begin near the end of one chunk and continue into the next. Overlap ensures that the explanation is preserved intact in at least one

The default values are conservative and heuristic. Smaller chunks improve precision; larger chunks preserve context. The right value depends on the corpus and the queries. For research papers in the Session 7 corpus, 400 words covers roughly two paragraphs and works well for the four test queries that read from indexed documents.

The honest limitation is that the chunker does not understand semantic boundaries. A chunk can begin mid-sentence and end mid-equation. The model receives the partial fragments and embeds them anyway. The embeddings are usually good enough; sometimes they are not. Session 8 introduces semantic chunking, in which the boundaries are placed by an LLM-aware boundary detector that knows about sentences, paragraphs, and section breaks. The sliding window stays in this session because it is simple, deterministic, and easy to read in source code.

The Memory upgrade

The Memory service in Session 6 stored items in a single JSON file under `state/memory.json` and read them through a keyword overlap scoring function. The same file remains the source of truth in Session 7. Items now carry an optional `embedding: list[float] | None` field. Items of kind `fact`, `preference`, and `tool_outcome` populate the field at insertion time by calling

```
class MemoryItem(BaseModel):
    id: str
    kind: Literal["fact", "preference", "tool_outcome", "scratchpad"]
    keywords: list[str]
    descriptor: str
    value: dict
    artifact_id: str | None = None
    embedding: list[float] | None = None      # new in Session 7
    source: str
    run_id: str
    goal_id: str | None = None
    confidence: float = 1.0
    created_at: datetime
```

A second persistence file appears alongside `state/memory.json`. The FAISS index is written to `state/index.faiss` and the parallel identifier list to `state/index_ids.json`. The three files together constitute the durable memory of the agent. Clearing the agent's state means deleting all three.

The read path is the load-bearing change. It is implemented as vector first, with keyword as the fallback:

The vector path returns first. The keyword fallback fires when the gateway is unreachable, when no items in the corpus carry embeddings, or when the FAISS index is empty. In the eight worked queries of this session the fallback never fires after the first iteration of the first run, because by then at least one item carries an embedding. The fallback is a graceful-degradation mechanism rather than a co-equal retrieval path.

Two write paths exist. The Session 6 `remember` function for ambiguous free-form content runs the LLM classifier as before, then computes an embedding on the classified descriptor before persisting. The Session 6 `record_outcome` function for deterministic tool dispatches embeds the descriptor it constructs

by the document-indexing tool to write chunks directly with no classifier call; the embedding is still computed at insertion.

```
def add_fact(descriptor: str, *, value: dict, keywords: list[str],
             source: str, run_id: str, goal_id: str | None = None) → MemoryItem:
    embedding = _try_embed(descriptor, task_type="retrieval_document")
    item = MemoryItem(
        id=new_id("mem"),
        kind="fact",
        keywords=[k.lower() for k in keywords],
        descriptor=descriptor,
        value=value,
        embedding=embedding,
        source=source,
        run_id=run_id,
        goal_id=goal_id,
    )
    return _persist_item(item)
```

The `_persist_item` helper appends to `memory.json`, then appends to the in-process FAISS index, then writes the FAISS index back to disk. The persistence is synchronous; every write touches disk before returning. At the scale this session reaches, the disk cost is invisible.

One subtle property of the implementation is that the FAISS index is reloaded from disk on every call. The reasoning is that the MCP server runs as a subprocess of the agent. The MCP subprocess's `index_document` tool writes new chunks into Memory. Those writes update `memory.json` and `state/index.faiss` on disk. The agent process, on its next memory read, must see those writes. Caching the FAISS index in the agent process's memory would hide the MCP-subprocess writes. Reloading from disk on every call is a small cost in exchange for cross-process consistency. The decision is documented in code and is one of the implementation details Session 8 will revisit when parallel

The gateway upgrade

The Session 5 gateway grew an `auto_route` mechanism, a router pool, and a per-provider failover ring. The Session 7 gateway adds one endpoint to that machinery. `POST /v1/embed` accepts a JSON body with a text string and an optional task-type label and returns a JSON body containing the embedding, the dimension, the provider that served the request, the exact model used, and the latency. The endpoint never sees agentic context: no system prompt, no tools, no prior history. It is the simplest endpoint in the gateway.

Two providers are configured. The first is Ollama running `omic-embed-text`, which produces 768-dimensional vectors and is the default. The second is Gemini's `gemini-embedding-001`, which natively produces 3072-dimensional Matryoshka vectors and is sliced to 768 by the gateway so both providers return vectors in the same semantic space. The slicing is configured at the gateway level through the `outputDimensionality` parameter on the Gemini request. Without the slice, the two providers would produce vectors that cannot be mixed in a single FAISS index. The slice is the cost of using the same index across providers.

The gateway's documentation states the consequence in bold near the top: the embedding model is fixed for the lifetime of any FAISS index built against it.

changing `EMBED_FALLBACK_MODEL` from `gemini-embedding-001` to a different model, or changing the configured output dimensionality, all silently invalidate every vector previously stored. The new vectors live in a different semantic space. Cosine similarities computed across the boundary are meaningless. The remedy is to delete the index files and rebuild from `memory.json`, which still contains the original text in each descriptor and chunk value.

The endpoint defends Gemini's free tier with a small rate budget and an exponential backoff. Ollama is local and uncapped. The agent code does not see these mechanics; it sees the `embed` call return a vector or raise an exception. The exception handling in Memory's `_try_embed` helper logs the failure and returns `None`, allowing the calling write to persist an item without an embedding. The keyword fallback in the read path then handles that item correctly. The failure mode is graceful and the agent continues to function.

The two new MCP tools

The agent's MCP server gains two tools. Each is a thin shim over the Memory module.

`index_document(path: str, chunk_size: int = 400, overlap: int = 80)` reads either a sandbox file (when `path` is a relative path) or an artifact (when `path`

descriptor begins with a bracketed source tag such as

[sandbox:papers/attention.md chunk 3/12] , which lets Perception recognise indexed chunks later when it scans memory hits. The function returns the number of chunks indexed.

`search_knowledge(query: str, k: int = 5)` runs the Memory read path filtered to `kind="fact"` items and returns up to `k` chunks with their provenance. Decision can call this tool when the goal asks the agent to answer from an already-indexed corpus. The tool returns chunk previews along with the source label and any metadata stored on the original `MemoryItem` .

The docstrings on both tools carry the distinction that Decision must observe to pick correctly between similar-looking alternatives:

```
@mcp.tool()
def index_document(path: str, chunk_size: int = 400, overlap: int = 80) → dict:
    """Chunk a sandbox file or artifact and write the chunks into Memory as
    fact records, where they become FAISS-searchable for later queries.
    Use this when the content must be searchable across later turns or runs.
    For one-shot inspection of a file's contents, use read_file."""

@mcp.tool()
def search_knowledge(query: str, k: int = 5) → list[dict]:
    """Vector search over previously indexed fact chunks. Use this rather
    than re-fetching or re-reading source files when Memory already
    contains indexed chunks for the topic."""
```

The placement of this guidance in the docstrings, rather than in any prompt, is one of the architectural lessons this session documents. The next section explains why.

The architectural principle: tool-blindness in Perception

Session 6 drew a clean boundary between the four roles. Perception decomposes the user's query into bounded goals. Decision picks the next action for one goal at a time. The MCP tool catalogue is part of Decision's input through the gateway's `tools=` parameter. Perception sees memory hits, the history, and the prior goal list. Perception does not see the tool catalogue.

The first implementation of the Session 7 tools regressed this boundary. When the model failed to select `index_document` on a goal that required indexing, the implementing process added a description of the tool catalogue to Perception's SYSTEM prompt, including specific tool names and usage hints. The test queries passed. The boundary was breached. Perception was now performing tool selection by name, which is Decision's job.

The regression is easy to slip into and worth understanding. Tool selection looked like a Perception problem because Perception emits the goals that Decision then dispatches. If the goal text said "use `index_document`" then Decision would pick `index_document` ; if the goal said "read the file" then Decision would pick `read_file` . The shortest path to passing the test was to make Perception emit the right goal text, which required Perception to know which tools existed. The shortest path was the wrong path.

Tool selection has two clean homes. The first is Decision's SYSTEM prompt, where general rules about choosing between similar-looking tools belong. The

tool-specific selection guidance because it co-locates the guidance with the tool definition; the guidance moves whenever the tool does, and the LLM sees it precisely when it is selecting tools.

The boundary was restored by reverting Perception's SYSTEM to intent-level language and absorbing the tool-selection guidance into Decision's SYSTEM and into the docstrings of `index_document` and `search_knowledge`. The eight test queries continued to pass after the revert. The architectural property of tool-blindness in Perception is now testable: a grep over `perception.py` confirms that no MCP tool name appears inside the SYSTEM string.

The pattern generalises. Roles in this architecture are responsibilities, and each responsibility has a vocabulary appropriate to it. Pushing the vocabulary of one role into another role to make a test pass produces an implementation that works under the queries in scope and fails to scale. When the agent grows from eleven tools to fifty, a tool catalogue inside Perception's SYSTEM becomes a budget-killing context bloat. A skill abstraction will eventually let Perception emit goals tagged with capability labels and let Decision filter the tool list by skill. The skill abstraction earns its complexity precisely because Perception will need a way to influence which tools are available without naming them. Holding the boundary in Session 7 keeps the design open to that abstraction.

A diagnostic discipline: examine what a

role sees

The architectural drift above was caught by a habit of inquiry worth naming. When a role appears to be looping, hallucinating, or making the wrong choice, the reflexive instinct is to add a rule to that role's SYSTEM prompt. The instinct fails often enough that it deserves to be replaced by a procedure.

The procedure begins with a question. Before adding any rule, ask what the role actually saw on the turn where it misbehaved. Reconstruct, from the source code that builds the prompt, the exact text the role received. Include every typed object that was rendered into the prompt: the memory hits, the recent history, the attached artifacts, the available tools, the prior goal list. The visible input to the role is the rendering of these objects at the upstream boundary. The rendering is the code path that runs between the typed object and the string the model sees. That code path can silently drop fields, truncate to a length that hides the decisive token, or coerce a structured payload into an opaque string.

The role behaves correctly given what it saw. Almost always. The question to ask is whether the rendering layer presented the role with the information needed to decide correctly. If yes, the bug lives in the role's SYSTEM or the model size. If no, the bug lives upstream in the rendering, and the fix belongs there. Adding a rule to the role's SYSTEM when the upstream rendering was the cause produces an implementation that works under the queries in scope and accumulates technical debt with every iteration.

The Session 7 work surfaced two clean examples of this pattern. The first concerned the cross-run mom's-birthday query. On the second run, Decision was supposed to answer from a memory hit that contained the date. The first attempt at a fix added a rule to Decision's SYSTEM that said "answer from memory when memory contains the date". The rule was unnecessary. The

and dropping the value. The descriptor in this case did not contain the date. Decision saw "mom's birthday remembered" but not "15 May 2026". The rule papered over the visible cause; the rendering function was the actual cause. The fix rendered the relevant `value` field in the prompt; the SYSTEM addition was removed.

The second example concerned the synonym-recall query against the indexed papers. On the third iteration of the run, Decision was looping on `search_knowledge` calls. The first attempt at a fix added an anti-loop rule to Decision's SYSTEM saying that the same tool with the same arguments must not be called twice in a row. The rule was wrong. The actual cause was that the memory hits returned by `search_knowledge` contained `value.chunk` fields with the chunk text, but `_format_hits` was rendering only the descriptors, which contained the source labels and chunk indices but not the chunk content. Decision saw "five chunks were found in the corpus" but had no way to tell what those chunks said. It concluded that the corpus must not contain the answer yet and asked for more chunks. The same five descriptors came back. The loop was rational given what Decision saw. The fix rendered `value.chunk` in `_format_hits` so Decision could read the content of the chunks attached to the goal. The loop disappeared without an anti-loop rule.

A general template for the procedure follows. The template is meant to be a checklist you can apply when they hit a role-level failure during their own assignment work.

- Step 1. Capture the failure. Save the full agent trace from the iteration where the misbehaviour appeared.
- Step 2. Identify the role. Which of Perception, Decision, Action, or Memory produced the output you consider wrong?
- Step 3. Reconstruct the input. From the source code that builds

to fields that may have been dropped by truncation or by the rendering function not addressing them.

Step 4. Ask the diagnostic question. Given the input you reconstructed, was the role's output rational?

If yes, the bug is upstream in the rendering. Locate the rendering function and fix it to expose the field the role needed.

If no, the bug is in the role itself. The role's SYSTEM is the right place to fix it, or the model size is too small for the task and the role needs a more capable worker through the gateway.

Step 5. Apply the fix at the right boundary. Do not patch the role's SYSTEM with a rule that compensates for a rendering bug. The rule will accumulate as the system grows and the rendering bug will surface again under a different query.

The procedure is short and the cases that follow it cleanly are common.

Worked queries

Eight queries exercise the Session 7 implementation. The first four are the Session 6 carryover queries; they verify that the new vector machinery has not

single-document indexing, cross-run persistence of the index, synonym recall through the vector path, and cross-document synthesis.

All eight queries were run verbatim against `agent7.py` after the architectural correction described above. The traces below are real and lightly cleaned for readability.

Query A. Shannon Wikipedia (artifact attach, carryover)

```
Fetch https://en.wikipedia.org/wiki/Claude_Shannon and tell me his
birth date, death date, and three key contributions to information
theory.
```

Three iterations. Decision picks `fetch_url`. Action pushes 263 KB of Wikipedia markdown to the artifact store. Perception attaches the artifact to the extraction goal in iteration two. Decision answers with the three contributions and the dates. The vector machinery is uninvolved; the answer comes from the attached artifact.

Query B. Tokyo activities and weather (multi-goal, memory carryover, carryover)

```
Find 3 family-friendly things to do in Tokyo this weekend.
Check Saturday's weather forecast there and tell me which one
is most appropriate.
```

Eight iterations. Perception decomposes into three goals: find activities, fetch the weather, select an activity given the weather. Memory carries the weather forecast from the second goal's tool outcome into the third goal's decision through the standard keyword path. The vector path participates but its results overlap with what keyword search would have returned.

```
Run 1: My mom's birthday is 15 May 2026. Remember that and create
       reminders for two weeks before and on the day.
Run 2: When is mom's birthday?
```

Run 1 takes four iterations. The user's query is passed through

`memory.remember`, the classifier extracts a `fact` item with the date in both the descriptor and the value, and two `create_file` calls produce reminder files in the sandbox. The fact item carries an embedding.

Run 2 takes three iterations and zero tool calls. The agent starts with empty in-process state. Memory's read on iteration one runs the query "When is mom's birthday?" through the gateway's embed endpoint, FAISS returns the fact item from run 1, Perception sees the date in the rendered hit, and Decision answers from memory.

Query D. Asyncio research (multi-source synthesis, carryover)

```
Search for "Python asyncio best practices", read the top 3 results,
and give me a short numbered list of the advice they agree on.
```

Six iterations. The agent searches, fetches three pages, and synthesises the common advice. Vector retrieval does not contribute; the three pages are attached to the synthesis goal as artifacts produced inside the same run.

Query E. Single-document index and extract

```
Index the file papers/attention.md and tell me what the three key
contributions of the Transformer architecture are according to this
paper.
```


iteration one. Action reads the markdown file from the sandbox, chunks it into eleven 400-word windows with 80-word overlaps, embeds each chunk, and writes the chunks as fact items. Iteration two finds Perception scanning the memory hits, recognising the indexed chunks by their

[sandbox:papers/attention.md ...] descriptors, and emitting an attach hint on the answer goal. Decision calls `search_knowledge` with a focused query and receives chunks describing self-attention, parallel computation, and positional encoding. The answer cites all three.

This is the simplest demonstration of the new tools. Indexing happens once. Retrieval happens once. The pattern composes naturally with everything else the agent already does.

Query F. Cross-run document recall (FAISS persistence)

```
Run 1: Index every .md file under papers/. Confirm how many chunks
      were indexed in total.
Run 2 (fresh process, persisted state):
      Across the papers I have indexed, what do they say about
      chain-of-thought reasoning?
```

Run 1 takes eleven iterations. Perception's first iteration emits a discovery goal that calls `list_dir` to enumerate the directory. The directory contains five paper files. Iteration two finds Perception appending five new goals at the end of the goal list, one per concrete file, and re-appending the original count-report goal as the final step. The append-after-discovery mechanism, established in Session 6's Perception SYSTEM, handles this gracefully. The five `index_document` calls run on iterations three through seven. Iteration eight reports the chunk count (fifteen chunks across the five papers). Iterations nine through eleven finalise.

observes that memory hits contain fact items with chunk descriptors, recognises the indexed corpus, and emits a single `search_knowledge` goal followed by a synthesis goal. Three iterations total. The agent answers from the persisted corpus without re-fetching any paper.

The architectural point lands here. The same agent code running in a different process arrives cold and reads its memory off disk. The FAISS index file on disk is the medium that crosses the process boundary. The `state/` directory between the two runs contains `memory.json` (twenty-three items, fifteen of them chunk facts), `index.faiss` (twenty-three vectors at dimension 768), and `index_ids.json` (twenty-three identifier strings in insertion order).

Query G. Synonym recall (vector beats keyword)

```
Across these papers, how do they handle the credit assignment
problem?
```

Four iterations. The phrase "credit assignment" appears in none of the indexed chunks; a grep over the five paper files returns zero hits. Pure keyword search returns nothing. The vector path, given an embedding of the query, surfaces chunks from four of the five papers that discuss conceptually related ideas: backpropagation through reasoning steps in the chain-of-thought paper, reward shaping in the DPO paper, the role of intermediate signals in the ReAct paper, and the parameter-efficient credit distribution argument in the LoRA paper. The synthesis goal pulls these four threads into an answer that attributes each claim to its source.

This is the strongest pedagogical demonstration in the set. Vector retrieval performs a search that keyword retrieval cannot. The architectural argument for embeddings in this session rests on this query.

Compare how the ReAct paper and the Chain-of-Thought paper differ in their treatment of intermediate reasoning.

Three iterations. `search_knowledge` returns chunks from both papers. Decision reads the attached chunks and produces a comparison that distinguishes ReAct's interleaving of reasoning and tool actions from the chain-of-thought paper's emphasis on linear stepwise reasoning. The agent attributes each claim correctly.

The eight queries together cover the architectural surface that Session 7 introduces. The state on disk after the F-run-1 to F-run-2 sequence is documented in `S7/NOTES_RUNS.md` and is reproducible by running the same eight queries in the same order.

State on disk and what crosses the process boundary

The persistence story is worth making explicit because it is what enables query F.

After indexing the five-paper corpus, the agent's `state/` directory contains

```
state/  
  memory.json      24 items: 15 chunk facts, 7 tool_outcomes, 2 classifier fac  
  index.faiss      23 vectors at dimension 768  
  index_ids.json   23 identifier strings, one per row of the FAISS index
```

The discrepancy between the twenty-four items in `memory.json` and the twenty-three vectors in the FAISS index is the scratchpad item that the user query produces; scratchpad items skip embedding by design. The remaining twenty-three items all carry vectors and appear in the FAISS index.

Both the agent process and the MCP subprocess write to these files. The agent's `memory.remember` and `memory.record_outcome` calls append to `memory.json` and to the FAISS index during the agent's main loop. The MCP subprocess's `index_document` tool, invoked through Decision via Action, runs in a separate Python process and writes to the same files. The agent's next memory read sees both kinds of writes because the index is reloaded from disk on every call.

The cross-process contract is simple. Whatever lives on disk after a write is what the next read sees. Caching the FAISS index in memory anywhere along the chain would break this contract. The Session 7 implementation makes the trade explicit: a few milliseconds of disk read per memory query in exchange for cross-process consistency without locking or shared-memory machinery.

A query against a process that has never been run before but starts with a populated `state/` directory works correctly. The agent has no in-process state to lose. It reads its memory off disk on iteration one. This is the property query F-run-2 exercises and the property you need to understand before you trust the agent to persist anything important.

Honest design choices for Session 7

The first is that vector retrieval has no hybrid sparse partner. Production retrieval systems run a sparse retriever (BM25 or learned-sparse) and a dense retriever in parallel and combine the ranked lists with Reciprocal Rank Fusion. The current session uses dense retrieval alone. Hybrid retrieval will arrive in a later session.

The second is that chunking is heuristic. The sliding window splits documents at arbitrary word boundaries. Semantic chunking, which uses an LLM-aware boundary detector to respect sentences, paragraphs, and section breaks, will arrive in Session 8.

The third is that the FAISS index is reloaded from disk on every call. The cost is small at the scale a single student will reach during their assignment. At higher scale, a memory-mapped index with file-modification-time invalidation, or an inter-process lock, becomes the right pattern. The session does not attempt to model that scale.

The fourth is the fixed-embedding-model constraint. The gateway pins the embedding model to ensure that all vectors in the FAISS index live in the same semantic space. Changing the model silently invalidates every vector previously stored. The gateway README states this in bold; the session text restates it. Students who try to substitute a different model mid-project will lose all their indexed knowledge silently. The remedy is to delete the FAISS files and rebuild

Each of these simplifications has a forward pointer to the session that addresses it properly. The deferrals are deliberate. Adding all of them in this session would push the cognitive load past where students can integrate the changes with what they built in Session 6.

Forward pointers

Session 8 introduces multi-agent DAG architecture and three retrieval upgrades that are natural extensions of what lands here.

Semantic chunking replaces the sliding window in `index_document`. The chunk boundaries are placed by an LLM-aware detector that recognises sentence boundaries, paragraph breaks, and structural markers. The chunker becomes its own small module with its own typed contract.

Hybrid retrieval with Reciprocal Rank Fusion combines the dense path from Session 7 with a sparse path implemented either as BM25 or as a learned-sparse model. The Memory read function gains a second retriever and a fusion step. The interface that the rest of the agent sees does not change.

Parallel fan-out at the agent loop level addresses the inefficiency surfaced by query F-run-1, which dispatches five `index_document` calls sequentially across eleven iterations. A DAG-based agent overcomes these five calls as parallel

nodes that execute concurrently inside an `asyncio.TaskGroup`. The eleven-iteration run compresses to four.

Skills arrive when the tool catalogue grows past what fits naturally inside Decision's prompt. A skill is an enumerated capability label that Perception can attach to a goal. Decision is invoked with the goal and a tool subset filtered by the skill. The skill abstraction is the architectural answer to the Perception tool-blindness rule established in this session: Perception influences which tools are available through a coarse label, never through specific tool names. The skill abstraction will land alongside the multi-agent architecture in Session 8.

Cross-encoder reranking is a candidate for a session beyond Session 8. The reranker is a small encoder that takes a query and a candidate chunk and produces a relevance score, used as a second-stage ranker over the top candidates returned by hybrid retrieval. The implementation is small but its placement in the architecture deserves its own treatment.

Knowledge-graph augmented retrieval and entity-level memory consolidation are further extensions, mentioned here only as forward-pointers.

Code

[C6 and C7 Code - LM October V7](#)

Assignment

Build something that uses RAG well, and prove the architecture is intact.

1. Pass the eight base queries (A through H) from this session. Verbatim, within the iteration bounds named alongside each.
2. Build a real RAG application over a corpus of fifty or more items. Pick a path:
 - a Chrome plugin that indexes pages you visit;
 - a desktop or cloud app over documents you choose;
 - any application that needs retrieval to work.
3. Design five queries against your corpus. Each must answer correctly with the index and fail without it. At least two must require semantic recall, where the query's words do not appear in the chunks that answer it.

Architectural rules carry over from this session. The grep test on Perception's SYSTEM stays the gate: zero MCP tool names inside. Adding a new MCP tool for a file format your corpus needs is welcome; its docstring is where Decision learns about it.

Submit a GitHub repository (README with the corpus manifest, the eight base traces, and the five custom traces with their no-corpus comparison), and a short video.

[Transcript](#)

Video

Studio

EAG V3 Session 7 Studio

The Admin



Watch on

GMeet

The Admin



PREVIOUS

Session 6 - Agentic Architecture